

2026 MARKET STACK · BACKEND TRACK

Java Interview Prep — 2 Years' Experience

A working reference of what's actually being asked right now: core language, the current LTS features, Spring Boot 3, and the surrounding cloud/data stack. Each entry is deliberately brief — enough to recall the shape of the answer, not a textbook chapter.

- 01 Core Java 02 Modern Java (17 / 21 / 25) 03 Concurrency & Threads 04 JVM & Memory
- 05 Spring Core & Boot 06 Building REST APIs 07 Data & Persistence 08 Security 09 Microservices
- 10 Messaging & Caching 11 Containers & CI/CD 12 Testing 13 Patterns & Principles
- 14 System Design Basics 15 Worth Knowing in 2026

Calibrate your depth. At 2 years, interviewers expect solid fundamentals, real hands-on Spring Boot experience, and the ability to reason about trade-offs — not JVM-tuning war stories or staff-level distributed-systems design. Depth beats breadth on the topics tagged **frequent**; the rest just needs "I know what it is and when I'd reach for it."

01 Core Java

Still the first 20 minutes of almost every interview — expect rapid-fire fundamentals before anything framework-specific.

OOP Pillars **frequent**

Encapsulation, inheritance, polymorphism, abstraction — be ready to explain each with a code example, not just the definition.

Collections Framework **frequent**

`ArrayList` vs `LinkedList`, `HashMap` internals (buckets, treeification after 8 collisions), `HashSet` vs `TreeSet`, when to reach for `ConcurrentHashMap`.

`equals()` & `hashCode()`

The contract between them, why breaking it corrupts hash-based collections, and how records

Exception Handling

Checked vs unchecked, try-with-resources and `AutoCloseable`, and why swallowing exceptions

generate both for free.

silently is a common review flag.

String Pool & Immutability

Why `String` is immutable, interning, and `==` vs `.equals()` gotchas.

Generics & Wildcards

Type erasure, bounded types (`<T extends X>`), and `? super` / `? extends` (PECS rule).

Access Modifiers & Packages

public/protected/default/private in practice, and how module boundaries (JPMS) rarely come up but are good to name-check.

02 Modern Java — 17, 21 & 25

17 and 21 are the LTS versions most shops actually run in production; 25 (Sept 2025) is the newest LTS and increasingly what greenfield roles ask about.

Streams & Lambdas frequent

`map/filter/reduce`, collectors, and when a plain for-loop is actually more readable than a stream one-liner.

Optional

Used to avoid null checks at API boundaries — interviewers often probe misuse, like calling `.get()` without checking presence.

Records 17+

Immutable data carriers that auto-generate constructor, accessors, `equals` / `hashCode` / `toString` — the default choice for DTOs now.

Sealed Classes 17+

Restrict which classes can implement/extend a type, enabling exhaustive pattern matching without a default branch.

Pattern Matching for switch 21

Switch expressions that match on type and deconstruct records directly in the case label.

Virtual Threads frequent 21

Project Loom's lightweight threads — millions of cheap threads instead of pooled platform threads, aimed squarely at blocking I/O-heavy services. Expect at least one question on this if the role touches Spring Boot 3.2+.

Text Blocks 17+

Multi-line string literals (`"""`) — mostly relevant for embedded SQL/JSON in code.

03 Concurrency & Threads

Less about writing raw threads, more about knowing the tools that manage them for you.

Executor Framework frequent

Thread pools via `ExecutorService`, fixed vs cached vs scheduled pools, and why unbounded pools are a production risk.

CompletableFuture

Composing async work with `thenApply` / `thenCompose`, and combining multiple futures.

synchronized & Locks

Intrinsic locks vs `ReentrantLock`, and when a lock is overkill compared to a concurrent collection.

Race Conditions & Deadlocks

Be ready to spot a race condition in a code snippet and explain the fix — a very common whiteboard exercise.

Virtual Threads vs Platform Threads

Virtual threads solve the "thread-per-request is expensive" problem without rewriting code in a reactive style.

04 JVM & Memory

Shallow-but-correct is the bar at 2 years — know the model, not the tuning flags.

Heap vs Stack

Objects live on the heap, method frames and local primitives on the stack — the usual source of `StackOverflowError` vs `OutOfMemoryError` questions.

Garbage Collection

Generational GC concept (young/old gen), and that G1 is the default collector; ZGC/Shenandoah exist for low-pause needs.

Class Loading

Common Memory Leaks

Bootstrap → platform → application classloader delegation — rarely deep-dived, good to recognize by name.

Static collections that keep growing, unclosed resources, and listeners that are never deregistered.

05 Spring Core & Boot

The single biggest chunk of a Java backend interview at this level. Spring Boot 3.x (Jakarta EE 9+, Java 17 baseline) is the current standard.

IoC & Dependency Injection frequent

Spring owns object creation and wiring; constructor injection is the preferred style over field injection for testability.

Bean Lifecycle & Scopes

Singleton (default) vs prototype vs request/session scope, and `@PostConstruct` / `@PreDestroy` hooks.

Auto-configuration & Starters

How `spring-boot-starter-*` dependencies trigger conditional bean registration behind the scenes.

Stereotype Annotations

`@Component`, `@Service`, `@Repository`, `@Controller` — functionally similar, semantically distinct, and `@Repository` also translates persistence exceptions.

AOP Basics

How `@Transactional` and logging aspects work via proxies — and why calling an annotated method from within the same class bypasses the proxy.

Profiles & Configuration

`application.yml` per environment, `@Value` vs `@ConfigurationProperties`.

06 Building REST APIs

Where "I've built things" gets proven — expect a live-coding or design-a-controller exercise.

@RestController & Mapping frequent

Bean Validation

`@Valid` with `@NotNull` / `@Size` etc. on DTOs, and how violations surface as 400s.

`@GetMapping` / `@PostMapping` , path vs query vs request-body params, and correct HTTP status code usage.

Global Exception Handling frequent

`@ControllerAdvice` + `@ExceptionHandler` to turn exceptions into consistent error responses instead of leaking stack traces.

API Versioning & Docs

URI or header-based versioning basics, and springdoc-openapi for Swagger UI generation.

DTO Pattern

Never expose JPA entities directly over the wire; mapping via MapStruct or manual constructors is the norm.

07 Data & Persistence

JPA/Hibernate questions are where candidates most often get caught out on shallow understanding.

Spring Data JPA frequent

Repository interfaces, derived query methods, and `@Query` for JPQL/native SQL.

Entity Relationships

`@OneToMany` / `@ManyToOne` , and owning side vs inverse side — a classic "explain mappedBy" question.

Lazy vs Eager Loading frequent

Default fetch types per relationship, and `LazyInitializationException` when accessed outside a session.

The N+1 Problem frequent

One query to fetch a list, then N more for each association — fixed with `JOIN FETCH` or entity graphs. Very commonly asked.

Transactions & ACID

`@Transactional` propagation and isolation levels, and what rolls back on a checked vs unchecked exception by default.

Schema Migrations

Flyway or Liquibase for versioned, repeatable schema changes instead of manual DDL.

SQL Fundamentals

Joins, indexing basics, and why an unindexed `WHERE` clause on a large table is a red flag.

08 Security

Even non-security roles get asked the basics of how a request gets authenticated.

Spring Security Filter Chain

Requests pass through a chain of filters before hitting a controller — authentication happens before authorization.

JWT Authentication frequent

Stateless auth via signed tokens, what's typically in the payload, and where refresh tokens fit in.

OAuth2 / OIDC Basics

Authorization Code flow at a conceptual level — enough to explain "login with Google" style flows.

Password Hashing

BCrypt (or Argon2) with salting — never store plaintext or use fast general-purpose hashes like MD5/SHA-1 for passwords.

09 Microservices & Distributed Systems

Most 2 YOE roles want awareness here, not deep architecture ownership.

Monolith vs Microservices

Trade-offs: independent deployability and scaling vs added operational and network complexity.

Service Discovery

Eureka or Consul let services find each other by name instead of hardcoded hosts.

API Gateway

Spring Cloud Gateway as the single entry point for routing, rate limiting, and auth.

Circuit Breaker frequent

Resilience4j — stop calling a failing downstream service and fail fast instead of cascading the failure.

Centralized Config

Distributed Tracing

Spring Cloud Config for externalized, environment-specific configuration across services.

Micrometer Tracing (successor to Sleuth) + Zipkin/Jaeger to follow a request across service boundaries.

10 Messaging & Caching

Kafka experience is one of the most commonly screened-for skills on job postings right now.

Kafka Basics frequent

Topics, partitions, producers/consumers, and consumer groups for parallel, ordered-per-partition processing.

RabbitMQ Basics

Exchange/queue/binding model — the more common choice for traditional task-queue style messaging.

Redis Caching frequent

`@Cacheable` / `@CacheEvict` in Spring, and cache invalidation as the perennially hard problem.

Event-Driven Basics

Publishing domain events instead of direct service-to-service calls to reduce coupling.

11 Containers, Cloud & CI/CD

Table stakes for "can this person ship" — most teams expect familiarity even without deep DevOps ownership.

Docker frequent

Writing a Dockerfile for a Spring Boot app, multi-stage builds to keep images small, and basic image/layer concepts.

Kubernetes Basics

Pods, deployments, and services at a conceptual level — enough to describe how a Java app gets deployed and scaled.

CI/CD Pipelines

GitHub Actions or Jenkins running build → test → package → deploy on every push.

Build Tools

Maven vs Gradle — dependency management, lifecycle phases, and multi-module project basics.

12 Testing

A "how do you test this" follow-up is almost guaranteed after any coding exercise.

JUnit 5 frequent

Annotations (`@Test` , `@BeforeEach`), assertions, and parameterized tests.

Mockito

Mocking dependencies to isolate the unit under test — `when/thenReturn` , `verify` .

Spring Boot Test Slices

`@SpringBootTest` for full context, `@WebMvcTest` / `@DataJpaTest` for faster, narrower slices.

Testcontainers frequent

Spins up a real Postgres/Kafka in Docker for integration tests instead of mocking the database — now the de facto standard over H2.

Test Pyramid

Many fast unit tests, fewer integration tests, a handful of end-to-end tests — know why the ratio matters.

13 Design Patterns & Principles

Asked less as trivia now, more as "why did you structure the code this way."

SOLID frequent

The five principles — Single Responsibility and Dependency Inversion come up most in Spring codebases specifically.

Singleton, Factory, Builder

Spring beans are singletons by default in practice; Builder shows up constantly for constructing complex DTOs/entities.

Strategy & Observer

Strategy maps naturally onto swapping implementations via DI; Observer underlies Spring's event publishing model.

DRY, KISS, YAGNI

Less "define these," more "does your code demonstrate them" during a live-coding round.

14 System Design Basics

Light-touch at this level — the goal is showing you can reason about scale, not designing Netflix.

Horizontal vs Vertical Scaling

Adding more instances vs a bigger box, and why stateless services make horizontal scaling straightforward.

Load Balancing

Round-robin vs least-connections at a conceptual level, sitting in front of multiple service instances.

Caching Strategy

Cache-aside as the pattern you'll actually be asked to reason through: read miss → load → populate cache.

CAP Theorem

Consistency, Availability, Partition tolerance — pick two; know where your database of choice lands.

15 Worth Knowing in 2026

Newer additions to the ecosystem — not core to most interviews yet, but increasingly showing up as "have you looked at" questions.

Spring AI

Spring's abstraction over LLM providers (chat clients, RAG, vector stores) — relevant if the team is building AI features on a Java backend.

GraalVM Native Image

Ahead-of-time compiled Java for near-instant startup and lower memory — increasingly used for serverless/container-dense deployments.

Java 25 LTS

The newest long-term-support release (Sept 2025); most production shops are still migrating from 17/21, but it's a fair "are you keeping up" question.

Structured Concurrency

A preview API for treating a group of related virtual-thread tasks as a single unit of work with unified cancellation and error handling.

Use this as a checklist, not a script: for each card, you should be able to say the one-liner out loud and back it with a project you actually built it in. Depth on the **frequent**-tagged items matters more than coverage of everything else.